Offline reference snippets

docs/research/drills/drills.py, verbatim -- 93 (b) "if internet is cut"

1. IoU (+ vectorised matrix)

```
def iou(a, b) -> float:
    iw = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    ih = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = iw * ih
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union if union > 0 else 0.0

def iou_matrix(A, B) -> np.ndarray:
    A, B = np.asarray(A, float).reshape(-1, 4), np.asarray(B, float).reshape(-1, 4)
    lt = np.maximum(A[:, None, :2], B[None, :, :2])
    rb = np.minimum(A[:, None, 2:], B[None, :, 2:])
    inter = np.clip(rb - lt, 0, None).prod(-1)
    area = lambda X: (X[:, 2] - X[:, 0]) * (X[:, 3] - X[:, 1])
    union = area(A)[:, None] + area(B)[None, :] - inter
    return np.where(union > 0, inter / np.maximum(union, 1e-12), 0.0)
```

Test to state aloud: union-zero guard returns 0.0, not NaN; matrix broadcasts (N,1,2)/(1,M,2).

2. NMS

```
def nms(boxes, scores, thr: float = 0.5) -> list[int]:
    boxes, scores = np.asarray(boxes, float).reshape(-1, 4), np.asarray(scores, float)
    order, keep = np.argsort(-scores, kind="stable"), []
    while order.size:
        i = order[0]
        keep.append(int(i))
        ious = iou_matrix(boxes[i:i + 1], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]
    return keep
```

Test to state aloud: sort by score descending; only IoU strictly over the threshold is suppressed.

3. Greedy IoU matching with gate

```
def greedy_assign(cost, max_cost: float = np.inf):
    cost = np.asarray(cost, float)
    pairs, used_r, used_c = [], set(), set()
    for k in np.argsort(cost, axis=None, kind="stable"):
        i, j = divmod(int(k), cost.shape[1])
        if cost[i, j] > max_cost:
            break
        if i not in used_r and j not in used_c:
            pairs.append((i, j)); used_r.add(i); used_c.add(j)
    return pairs
```

Test to state aloud: sorts the whole cost matrix once; a pair above max_cost stops the loop, not just that pair.

4. Kalman constant-velocity predict/update (box)

```
    def predict(self):
        h = self.x[3]
        Q = np.diag(np.square([h / 20, h / 20, 1e-2, h / 20, h / 160, h / 160, 1e-5, h / 160]))
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + Q

    def update(self, xywh):
        x, y, w, h = xywh
        z = np.array([x + w / 2, y + h / 2, w / h, h])
        hh = self.x[3]
        R = np.diag(np.square([hh / 20, hh / 20, 1e-1, hh / 20]))
        S = self.H @ self.P @ self.H.T + R
        K = self.P @ self.H.T @ np.linalg.inv(S)
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(8) - K @ self.H) @ self.P
```

Test to state aloud: state is (cx, cy, w/h, h, and their 4 velocities); noise scales with box height h.

5. Two-stage IoU tracker (birth/death)

```
def track(frames, hi=0.6, lo=0.1, iou_thr=0.3, max_age=30):
    tracks, next_id, out = [], 1, []          # track = dict(id, box, age)
    for f, (boxes, scores) in enumerate(frames):
        boxes, scores = np.asarray(boxes, float).reshape(-1, 4), np.asarray(scores, float)
        high = list(np.flatnonzero(scores >= hi))
        low = list(np.flatnonzero((scores >= lo) & (scores < hi)))
        free, unmatched_high = list(range(len(tracks))), high
        for stage, det in ((1, high), (2, low)):   # 1: confident; 2: leftovers vs weak
            pairs = []
            if free and det:
                tb = np.array([tracks[t]["box"] for t in free])
                pairs = hungarian(1 - iou_matrix(tb, boxes[det]), 1 - iou_thr)
            for ti, di in pairs:
                t = tracks[free[ti]]
                t.update(box=boxes[det[di]], age=0)
```

```
                out.append((f, t["id"], t["box"].tolist()))
            if stage == 1:
                unmatched_high = [det[d] for d in range(len(det)) if d not in {di for _, di in
pairs}]
            free = [t for k, t in enumerate(free) if k not in {ti for ti, _ in pairs}]
        for t in free:
            tracks[t]["age"] += 1
        for d in unmatched_high:              # birth only from confident detections
            tracks.append({"id": next_id, "box": boxes[d], "age": 0})
            out.append((f, next_id, boxes[d].tolist())); next_id += 1
        tracks = [t for t in tracks if t["age"] <= max_age]   # death
    return out
```

Test to state aloud: birth only from confident (high-score) unmatched detections; death is age > max_age.

6. MOTA + IDSW, and IDF1 via Hungarian

```
def mota_idf1(gt, pred, thr: float = 0.5):
    fn = fp = idsw = n_gt = 0
    last, prev = {}, {}                        # gt id -> pred id: ever / previous step
    pair_count, gt_n, pr_n = {}, {}, {}
    for f in sorted(set(gt) | set(pred)):
        g, p = gt.get(f, {}), pred.get(f, {})
        gi, pi = list(g), list(p)
        n_gt += len(gi)
        for k in gi: gt_n[k] = gt_n.get(k, 0) + 1
        for k in pi: pr_n[k] = pr_n.get(k, 0) + 1
        if not gi or not pi:                   # empty side: all FN or all FP
            fn += len(gi); fp += len(pi)
            continue
        S = iou_matrix([g[k] for k in gi], [p[k] for k in pi])
        for a, b in zip(*np.nonzero(S >= thr - 1e-12)):
            pair_count[gi[a], pi[b]] = pair_count.get((gi[a], pi[b]), 0) + 1
        score = S + 1000 * np.array([[prev.get(a) == b for b in pi] for a in
gi]).reshape(S.shape)
        score[S < thr - 1e-12] = 0
        m = [(a, b) for a, b in hungarian(-score) if score[a, b] > 0]
        for a, b in m:
            if gi[a] in last and last[gi[a]] != pi[b]:
                idsw += 1
            last[gi[a]] = pi[b]
        prev = {gi[a]: pi[b] for a, b in m}
        fn += len(gi) - len(m); fp += len(pi) - len(m)
    G, P = sorted(gt_n), sorted(pr_n)
    C = np.array([[pair_count.get((a, b), 0) for b in P] for a in G], float).reshape(len(G),
len(P))
    idtp = sum(C[a, b] for a, b in hungarian(-C)) if C.size else 0.0
    mota = (n_gt - fn - fp - idsw) / max(1, n_gt)       # = 1-(FN+FP+IDSW)/N_gt; no GT: -FP
    tot = sum(gt_n.values()) + sum(pr_n.values())
    idf1 = 2 * idtp / tot if tot else 0.0
    return mota, idf1, idsw
```

Test to state aloud: MOTA = 1-(FN+FP+IDSW)/N_gt (no-GT case is -FP, not 0/0); IDF1 is one global Hungarian match on accumulated IoU>=0.5 pair counts, not a per-frame match.

7. Dwell rule with hysteresis

```
def loiter_alarms(in_zone, fps: float, t_on: float, t_off: float) -> list[int]:
    """Alarm frame indices. Dwell accumulates while inside; an exit shorter than
    t_off is a blip (dwell keeps counting); one alarm per episode."""
    alarms, dwell, out_run, fired = [], 0, 0, False
    on, off = round(t_on * fps), round(t_off * fps)
    for i, inside in enumerate(in_zone):
        if inside:
            dwell, out_run = dwell + 1, 0
            if dwell >= on and not fired:
                alarms.append(i); fired = True
        elif dwell:
            out_run += 1
            if out_run >= off:
                dwell, out_run, fired = 0, 0, False
    return alarms
```

Test to state aloud: dwell keeps counting through an exit shorter than t_off; fires once per episode, re-arms only after a real exit.

8. Percentile bootstrap CI + exact paired sign-flip p

```
def bootstrap_ci(d, n_boot: int = 10000, seed: int = 51, level: float = 95.0):
    d = np.asarray(d, float)
    m = d[np.random.default_rng(seed).integers(0, len(d), (n_boot, len(d)))].mean(1)
    return float(np.percentile(m, (100 - level) / 2)), float(np.percentile(m, 100 - (100 -
level) / 2))
```

```
def signflip_p(d) -> float:
    """Exact two-sided: fraction of all 2^n sign patterns with |mean| >= observed."""
    d = np.asarray(d, float)
    signs = np.array(np.meshgrid(*[[1, -1]] * len(d))).reshape(len(d), -1).T
    return float((np.abs((signs * d).mean(1)) >= abs(d.mean()) - 1e-12).mean())
```

Test to state aloud: bootstrap CI is percentile (not BCa) and too narrow at small n; sign-flip enumerates all 2^n sign patterns exactly.

9. DLT homography with Hartley normalisation

```
def _hartley(p):
    m = p.mean(0)
    s = np.sqrt(2) / np.mean(np.linalg.norm(p - m, axis=1))
    return np.array([[s, 0, -s * m[0]], [0, s, -s * m[1]], [0, 0, 1]])

def dlt_homography(src, dst, normalise: bool = True) -> np.ndarray:
    """src px -> dst ground metres from >= 4 correspondences. normalise=True is
    Hartley (centroid to origin, mean distance sqrt 2) and is what survsim does;
    normalise=False is the textbook DLT, identical only on exact points."""
    src, dst = np.asarray(src, float), np.asarray(dst, float)
    Ts, Td = (_hartley(src), _hartley(dst)) if normalise else (np.eye(3), np.eye(3))
    a = (Ts @ np.c_[src, np.ones(len(src))].T).T
    b = (Td @ np.c_[dst, np.ones(len(dst))].T).T
    rows = []
    for (x, y, _), (u, v, _) in zip(a, b):
        rows.append([-x, -y, -1, 0, 0, 0, u * x, u * y, u])
        rows.append([0, 0, 0, -x, -y, -1, v * x, v * y, v])
    Hn = np.linalg.svd(np.asarray(rows))[2][-1].reshape(3, 3)
    H = np.linalg.solve(Td, Hn @ Ts)            # denormalise: Td^-1 Hn Ts
    return H / H[2, 2]
```

Test to state aloud: normalise both point sets before building the design matrix; take the last right-singular vector, then denormalise Td^-1 Hn Ts.

10. argparse + pytest skeleton, and check.sh

```
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--in", dest="inp", required=True)
    p.add_argument("--out", required=True)
    args = p.parse_args()
    ...
if __name__ == "__main__":
    main()

import pytest
def test_empty_input():
    assert f([]) == []
def test_happy_path():
    assert f([...]) == [...]

#!/bin/sh
set -e
python -m pytest -q
python -m pyflakes . 2>/dev/null || python -m ruff check .
```

Test to state aloud: activate the venv first; run pytest -q (not -n auto) in an unfamiliar sandbox.